

Assignment-2

Part I: Short Answer / Reasoning Questions

Q1. Comparisons cannot be used with NULL as they get evaluated to UNKNOWN instead of TRUE. The statement WHERE phone_number ISNULL should be used instead. The WHERE clause only keeps rows where the condition is TRUE. Rows where the condition is FALSE or UNKNOWN get filtered out.

Q2. The word DISTINCT is redundant here because of the GROUP BY department operation, which produces one row per department and thus there cannot be duplicate departments. The keyword tries to remove duplicates from (department, COUNT (*)) but there will be none as different departments will be grouped automatically in one row e.g. ((Marketing, 50)).

Q3. If left join produces more rows, it means that there are customers in Customer table without any matching rows in the Order table. Inner join excludes them as it only takes entries with matching values in column.

Q4. In this query, department is under GROUP BY and salary is under SUM – an aggregate function. However, employee_name is under neither and thus the query is invalid. Therefore, due to multiple names under a department the function would return any arbitrary value (undefined behaviour) if such a query was allowed.

Q5. WHERE cannot use aggregate functions like AVG because it filters the rows before grouping. Functions like AVG can only be used after grouping. We correct this query using HAVING:

```
SELECT department, AVG (salary)
FROM Employee
GROUP BY department
HAVING AVG (salary) > 80000;
```

Q6. For (a) a non-correlated subquery is better because the overall average product price is a single value which is independent of the other rows of the table. For (b) a correlated subquery is better because the average salary depends

on the employee's department. Each outer row (employee) must trigger a subquery restricted to that department. A non-correlated subquery is evaluated once and the result is reused for all the rows of the table. A correlated subquery is evaluated for each row of the outer query, which can result in multiple executions for large tables.

Part II: Long Answer Question

(a) The query can be written as:

```
SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
LEFT JOIN Order o
ON c.customer_id = o.customer_id
WHERE o.order_id ISNULL;
```

LEFT JOIN is used to access all the customers (INNER JOIN only reports those customers who have placed an order). Then we filter the customers without orders (having order_id) as NULL using WHERE. We did not use HAVING as no aggregate functions are required.

(b) The query can be written as:

```
SELECT r.restaurant_id, r.name,
AVG (order_total.order_value) AS avg_order_value
FROM restaurants r
INNER JOIN orders o ON r.restaurant_id = o.restaurant_id
INNER JOIN (
SELECT
order_id,
SUM (price * quantity) AS order_value
FROM order_items
GROUP BY order_id)
AS order_total ON o.order_id = order_total.order_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT (DISTINCT o.order_id) > 5;
```

We used HAVING to take averages after grouping (aggregate value), INNER JOIN is used because we are only concerned with restaurants with orders and orders with items.

Part III: Normalization

Q1. student_id -> student_name

course_id -> course_name, instructor_id

instructor_id -> instructor_name, instructor_office

(student_id, course_id) -> grade, enrolment_date

Q2. The table has a primary key, namely (student_id, course_id). No data types are mixed in any of the columns. The columns of the table are not in the form of similar groups like course1, course2, course3 etc. and row order is not used to convey information. Therefore, the table is in 1NF.

Q3. student_id -> student_name

course_id -> course_name, instructor_id

The above partial dependencies violate 2NF because update anomalies can occur (if a student enrolls in another course/drops a course). Other possible anomalies are deletion and insertion anomalies. The issue here is that these components only depend on one part of the composite key, which leads to above anomalies.

Q4. The transitive dependency course_id -> instructor_id and instructor_id -> instructor_name, instructor office violates 3NF. This violates 3NF because non-key attributes depend on other non-key attributes via a chain. This can cause update anomalies e.g. if an instructor changes his office, then every course row would have to be updated.

Q5. Student (student_id, student_name)

Instructor (instructor_id, instructor_name, instructor_office)

Course (course_id, course_name)

Enrollment (student_id, course_id, grade, enrollment_date)

Here every non-key attribute depends only on the whole key, and nothing but the key. Also, partial and transitive dependencies have been removed.

Part IV: Design Justification (150–200 words)

In part III, since the data is split into multiple tables, running a query becomes harder as now we would have to JOIN depending on the type of query that we have to use. Before normalization, all the information about a student, course, instructor and enrolment was in one row and one scan

was sufficient to get the output. However, now the searches can be slower for large tables and the code becomes more complex as multiple JOINS have to be implemented. Thus, the reduction in anomalies comes at a performance cost.

In part II (b), the JOIN version makes the logic explicit that we have to compute averages once and then compare. The code is easier to read when explaining to others. The correlated subquery looks shorter but it hides the repeated evaluation. Both handle duplicates fine since AVG ignores them. But JOIN avoids subtle NULL issues because the derived table has one row per department, whereas correlated subqueries can sometimes misbehave if outer values are NULL. JOIN is faster as it computes average once, but they are recalculated for every row in the correlated subquery which leads to thousands of redundant calculations.